jigsaw-agile-community-rules

3位 Solution

Rank	3
Team	Sergio Manuel Papadakis
Author	Sergio Manuel Papadakis
Topic ID	613324
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613324
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、この魅力的なコンペを企画・開催してくださった Kaggle と Jigsaw ACRC に感謝を表したいと思います。

最も良かった提出を public にしました: notebook。

この解法は次のアイデアに基づいています

- 多くのルールが隠されていたため、推論時にモデルを学習させる Test-Time Training (TTT) に頼った。
- どの単一モデルよりも性能が高く、public と private の leaderboard 間のスコアの変動にも耐えられる、強いアンサンブルを構築した。
- すべてのモデルと最終アンサンブルは Public LB に対して検証した。
- 12時間の推論枠の中で、できるだけ多くのモデルを学習させた。
- 最終アンサンブルは2つのサブアンサンブルで構成した。
  - slow ensemble: LLM で構成。2xQwen2.5-instruct-14b、2xQwen2.5-instruct-7b、2xQwen3-instruct-14b、2xQwen3-instruct-8b。
  - fast ensemble: より高速なモデルで構成。2xbge-base-v1.5、Qwen3-embedding-0.6b、DeBERTa-v3-small。

学習方法

- 事前学習と TTT を行うため、すべてのルール、body/例、およびそれらに対応する既知のターゲットを使って新しいデータセットを構築した。
- subreddit 列は捨てた。事前学習と TTT の両方の段階で役に立たないことが分かった。
- 事前学習の段階は、この notebook を使って行った。

Slow ensemble

- LLM は、public データセットを使って1 epoch の学習で事前学習した。
- LLM の事前学習は zero-shot 形式で、次のプロンプトを使って行った。
  - System prompt: "You are given a comment on reddit. Your task is to classify if it violates the given rule. Only respond Yes or No.\nRule: {rule_str}"
  - User prompt: “1. `{example_body_str}` \nViolation:{answer_str}”
- 事前学習の段階の出力は LoRA の重みである。これらの重みは、後で TTT の段階の出発点を設定するために使った。このアプローチにより、すべての LoRA がすでにコンペのタスクと使用するプロンプトを知っていたため、TTT 中にモデルがより良い性能を達成するのに役立った。
- 事前学習と TTT では、LLM の学習に Unsloth ライブラリを使った。Unsloth のおかげで、GPU（T4）ごとに1つのモデルを置き、並列で学習を行えた。
- 推論は vLLM で行い、出力は最後のトークンの埋め込みとした。

- すべてのモデルを12時間の推論枠に収めるため、Qwen2.5-instruct-7b の2つの LoRA の重みをマージし、単一モデルとして推論した。同じアプローチを2つの Qwen3-instruct-7b モデルにも適用した。
- TTT 中、最終的な LoRA を使って学習データセットとテストデータセットの両方で推論を行った。出力は最後の "Violation:" トークンの埋め込みである。
- これらの埋め込みを使って、交差検証した古典的な機械学習（ML）モデル群（LightGBM、XGBoost、SVC）を学習した。その狙いは、学習データセット（TTT 中に使ったもの）を使ってこれらの ML モデルを学習させ、他のモデルからの対応する確率とアンサンブルできる確率を出力させることである。

Fast ensemble

- fast ensemble のモデルはすべて事前学習の段階を持たず、TTT 中にのみ学習した。

bge-base-en-v1.5:

- bge-base-en-v1.5 モデルは、public の notebook で提案されているように Triplet Loss を使って学習した。
- このモデルを2つのバージョンで学習した。1つは与えられたとおりのルールを使うもの（NormalRules）、もう1つはルールを反転させたもの（FlippedRules）である（たとえば、“do not give legal advice” と言う代わりに、ルールを否定して “give legal advice” にした）。
  - NormalRules 版: triplet loss の positive サンプルはルール違反のない例、negative サンプルはルール違反のある例、anchor はルールとした。
  - FlippedRules 版: 例を反転させた。triplet loss の positive サンプルはルール違反のある例、negative サンプルはルール違反のない例、anchor は反転したルールとした。
- 学習後、最終モデルを使ってデータセット内のすべての例と body の埋め込みを生成した。positive と negative の重心の埋め込みを計算し、最終的な埋め込みを次のように構成した。 final_embedding = concat([body_embeddings - positive_examples_centroid_embedding, body_embeddings - negative_examples_centroid_embedding])
- これらの埋め込みを使って、ルール違反を予測する新しい ML モデル群を学習した。最終的な予測は fast ensemble に入力した。

Qwen3 embedding 0.6b

- ArcFace loss を使ってルール違反を予測する LoRA を学習した。出力クラスは rule_violation と no_rule_violation の2つだけである（他の構成、たとえばルールごとに1クラスにするなど、より多くのクラスも試したが、私のテストではうまくいかなかった）。
- このモデルの入力には body だけを使った（'{rule}: {body}' を入力とする学習も試したが、スコアは改善しなかった）。
- mean pooling を使って、学習データセットとテストデータセットの両方のすべての body と例から埋め込みを抽出した。
- これらの埋め込みを使って、ルールごとにルール違反を予測する新しい ML モデル群を学習した。得られた予測は fast ensemble の入力として使った。

DeBERTa v3 small

- DeBERTa v3 small モデルの学習には cross entropy loss を使い、最初のトークン（[CLS] トークン）から埋め込みを抽出した。
- このモデルの入力には '{rule}[SEP]{body}' を使った。
- これらの埋め込みを使って、ルール違反を予測する新しい ML モデル群を学習した。
- ML モデルから得た確率を DeBERTa モデルの出力確率と平均した。最終的な予測は fast ensemble に入力した。

部分アンサンブルのスコア

Model	Public Score	Private Score
2xQwen2.5-instruct-14b	0.90828	0.90248
2xQwen3-instruct-14b	0.92571	0.92049
2xQwen2.5-instruct-7b	0.91024	0.90637
2xQwen3-instruct-8b	0.89598	0.88969
2xbge-base-v1.5	0.90952	0.89714
Qwen3-embedding-0.6b	0.88856	0.88021
DeBERTa-v3-small	0.89546	0.88946

補足Q&A

Shivam A. — 質問

おめでとうございます!! これらのモデルの leaderboard での個別の性能もありますか？

ShadowCoder — 質問

共有ありがとうございます。TTT 中に、なぜ最終的な LoRA を使って学習データセットとテストデータセットの両方で推論を行うのか知りたいです。つまり、TTT 中に、まずテストセットを実行してから学習セット + テストセットを使うのではなく、元のテストセットの代わりに学習セット + テストセットを直接使って TTT を行える、という意味ですか？

Sergio Manuel Papadakis — 回答

ありがとう @shivamreturns!!! サブアンサンブルの性能を要約に追加しました。私の notebook を使っていくつかテストしたい場合のために、事前学習したすべての LoRA の重みも public にしました。

Sergio Manuel Papadakis — 回答

ありがとう @fondrose、

Unsloth で TTT を行った後、最終的な予測（ルール違反の確率）を出すすべての古典的な ML モデルを学習するために、最終モデルを使って TTT の学習セット（violation または no_violation のどちらかでラベル付けされた例を含む）で埋め込みの推論を実行する必要がありました。

また、test.csv ファイルの body で推論を実行して埋め込みを抽出し（これを本来のテストデータセットと呼んでいます）、前のステップで学習したすべての ML モデルを評価する必要もありました。